

Automated Delivery of Session Summaries as Compiled PDFs: Integrating a LaTeX Paper-Writer Subagent with an Obsidian Vault

[Author placeholder]

September 16, 2026

Abstract

This paper documents the integration of a LaTeX paper-writing subagent with a personal Obsidian vault, enabling the automatic delivery of every chat-session summary as a compiled PDF research paper. The integration required two components: (i) a revision of the subagent’s output contract so that each paper is compiled with `latexmk -pdf` and archived, together with its $\text{\LaTeX}$ source, in a dedicated `session-summaries` folder inside the vault’s main container; and (ii) the installation of a lightweight $\text{\TeX}$ toolchain on a machine that previously had none. Because no Homebrew cask for TinyTeX exists, the official TinyTeX installer script was used, yielding TeX Live 2026 with `pdfTeX 3.141592653-2.6-1.40.29` installed under `~/Library/TinyTeX`; all binaries were symlinked into `/opt/homebrew/bin`, which is already on the shell’s `PATH`. Verification confirmed that `pdflatex`, `xelatex`, `latexmk`, and `tlmgr` resolve and that the common packages `amsmath`, `amssymb`, `graphicx`, `hyperref`, and `geometry` are available. A known caveat is that the agent host loads agent definitions at startup, so the revised contract becomes fully live only after the next restart; the present run was therefore instructed explicitly via the task prompt.

1 Introduction

Personal knowledge management systems such as Obsidian are increasingly used as the archival layer for the artifacts produced by AI-assisted workflows. In this setting, a recurring operational need is the reliable conversion of conversational session summaries into durable, human-readable documents. This paper reports on a concrete engineering exercise: integrating a dedicated *latex-paper-writer* subagent with the user’s Obsidian vault so that every session summary is delivered as a compiled PDF paper.

The motivating problem had two parts. First, the subagent’s output contract specified only that a self-contained $\text{\LaTeX}$ source file be produced; nothing guaranteed a compiled PDF, nor any delivery into the vault. Second, the target machine (Apple Silicon, macOS, zsh) had no $\text{\LaTeX}$ toolchain installed, so compilation was not possible at all. The research question addressed here is therefore twofold:

- (i) How should the subagent’s contract be revised so that each session summary is compiled and archived inside the vault in a predictable, versioned manner?
- (ii) How can a minimal, self-contained $\text{\TeX}$ toolchain be installed on a machine without administrative privileges for `PATH` modification, and verified end to end?

The remainder of this paper describes the vault structure and the agent system (Section 2), the contract revision and toolchain installation (Section 3), the verification results (Section 4), and the operational caveats (Section 5).

2 Background

2.1 The Obsidian vault

The user’s personal Obsidian vault is rooted at `~/Documents/vmain`. Its foundational container, `~/Documents/vmain/main/`, organizes all notes into category folders: `documents`, `projects`, `questions`, `quick notes`, `systems`, and `web-clips`. The integration described here introduces a new category folder, `session-summaries`, which holds the compiled PDF papers and their $\text{\LaTeX}$ sources, one pair per session, named `YYYY-MM-DD-topic-slug`.

2.2 The agent system

The subagent is defined in `~/.config/opencode/agents/latex-paper-writer.md` and registered in the swarm registry `~/.config/opencode/agent-groups.json`. The agent host loads agent definitions at startup; consequently, edits to the definition file take full effect only after the next restart of the host. The agent’s hard rules require that content, data, citations, and results never be fabricated; that inferred or uncertain material be marked with `% TODO` comments; that the standard `article` class with common packages be used; and that writes occur only inside the vault folder and the temporary build directory.

2.3 TinyTeX

TinyTeX is a lightweight, cross-platform $\text{\TeX}$ distribution designed for casual users and automated pipelines. On macOS it installs under the user’s home directory, which avoids the need for system-wide installation. The Homebrew cask `tinytex` does not exist, so installation must proceed via the official installer script hosted at yihui.org/tinytex/install-bin-unix.sh. The installed distribution is TeX Live 2026, providing pdfTeX 3.141592653-2.6-1.40.29.

3 Methods

3.1 Revision of the output contract

The agent definition was updated so that the output contract now requires the following delivery pipeline:

1. Write the $\text{\LaTeX}$ source to a temporary build directory (e.g., `/tmp` or the host’s temp directory), never directly into the vault.
2. Compile with `latexmk -pdf`, falling back to two passes of `pdflatex -interaction=nonstopmode` if `latexmk` is unavailable.

3. Create `~/Documents/vmain/main/session-summaries/` if it does not exist.
4. Copy the compiled PDF and the $\text{\LaTeX}$ source into that folder, named `YYYY-MM-DD-topic-slug.pdf` and `YYYY-MM-DD-topic-slug.tex`.
5. Remove auxiliary files from the build directory.

To make this pipeline executable autonomously, the `bash` permission for the agent was changed from `ask` to `allow`, so that the compiler can be invoked without interactive confirmation. The swarm registry’s skill description for the agent was updated in parallel to reflect the PDF-to-vault delivery behavior.

3.2 Toolchain installation

The machine had no $\text{\LaTeX}$ toolchain. The installation proceeded as follows:

1. The Homebrew cask `tinytex` was checked and found not to exist, ruling out the cask route.
2. The official TinyTeX installer script (yihui.org/tinytex/install-bin-unix.sh) was executed, installing TeX Live 2026 under `~/Library/TinyTeX`.
3. The installer could not modify `PATH` because that operation requires `sudo`. As a workaround, all binaries (`pdflatex`, `xelatex`, `latexmk`, `tlmgr`, and others) were symlinked into `/opt/homebrew/bin`, which is already on the shell’s `PATH`.

3.3 Verification

The installation was verified in two ways: (i) command resolution, confirming that `pdflatex`, `xelatex`, `latexmk`, and `tlmgr` all resolve from `/opt/homebrew/bin`; and (ii) package availability, confirming that the common packages `amsmath`, `amssymb`, `graphicx`, `hyperref`, and `geometry` are present in the distribution.

4 Results

All verification checks passed. The toolchain resolves correctly: `pdflatex` reports `pdfTeX 3.141592653-2.6-1.40. (TeX Live 2026)`, and `latexmk`, `xelatex`, and `tlmgr` are all reachable on `PATH`. The required packages are available, so the standard `article`-class template compiles without modification.

The vault folder `~/Documents/vmain/main/session-summaries/` was created and now contains the first artifact of the pipeline: the present paper, delivered as `2026-09-16-latex-paper-writer-vault-integration.pdf` together with its $\text{\LaTeX}$ source. This constitutes an end-to-end demonstration of the revised contract: source written to a temporary build directory, compiled with `latexmk -pdf`, archived into the vault, and auxiliary files cleaned up.

5 Discussion

The integration is operational, but one caveat deserves emphasis. The agent host loads agent definitions at startup, so the revised definition in `~/.config/opencode/agents/latex-paper-writer.md` becomes fully live only after the next restart of the host. The present run was therefore instructed explicitly through the task prompt, which carried the delivery instructions verbatim. Future sessions that spawn the agent after a restart will inherit the contract automatically.

Two design decisions are worth noting. First, the choice of TinyTeX over a full TeX Live installation reflects the requirement of a lightweight, user-level toolchain; the absence of a Homebrew cask made the official installer script the natural route. Second, the `PATH` symlink workaround into `/opt/homebrew/bin` avoids `sudo` entirely while keeping the binaries discoverable by the agent's shell; the trade-off is that the symlinks must be maintained if the distribution is upgraded or relocated.

6 Conclusion

This paper documented the integration of a LaTeX paper-writing subagent with an Obsidian vault, delivering every session summary as a compiled PDF. The output contract was revised to compile with `latexmk -pdf` and archive the PDF and source into `session-summaries` under the vault's main container; a lightweight TinyTeX toolchain (TeX Live 2026) was installed without `sudo` via the official installer script and symlinked onto `PATH`; and the pipeline was verified end to end with the present paper as its first artifact. The remaining operational step is a restart of the agent host so that the revised contract is loaded automatically for all future sessions.

References

- [1] Yihui Xie. *TinyTeX — A lightweight, cross-platform, portable and easy-to-maintain LaTeX distribution*. <https://yihui.org/tinytex/>. Accessed September 16, 2026.
- [2] Yihui Xie. *TinyTeX installation script for Unix-like systems*. <https://yihui.org/tinytex/install-bin-unix.sh>. Accessed September 16, 2026.
- [3] TeX Users Group. *TeX Live*. <https://tug.org/texlive/>. Accessed September 16, 2026.
- [4] Homebrew. *The Missing Package Manager for macOS (or Linux)*. <https://brew.sh/>. Accessed September 16, 2026.
- [5] Obsidian. *Obsidian — Sharpen your thinking*. <https://obsidian.md/>. Accessed September 16, 2026.
- [6] John Collins. *latexmk — Fully automated LaTeX document generation*. <https://ctan.org/pkg/latexmk>. Accessed September 16, 2026.